

Unit 05: Design

CONTENTS

Objectives

Introduction

- 5.1 What is a Software Design?
- 5.2 Design Process
- 5.3 Objectives of Software Design
- 5.4 Software Design Levels
- 5.5 Software Design Concepts
- 5.6 Coupling and Cohesion
- 5.7 Data Flow Diagram and Flow Chart
- 5.8 Flow Chart
- 5.9 Architectural Design
- 5.10 Component Based Design
- 5.11 Object-oriented design
- 5.12 Use Case Diagram, Class Diagram and Activity Diagram

Summary

Keywords

Self Assessment

Answer for Self Assessment

Review Questions

Further Reading

Objectives

- after studying this unit, you will be able to:
- design process and design concepts
- data flow diagram (DFD) and flow chart, architectural design
- use case diagram, class diagram, activity diagram

Introduction

Software design is a problem-solving and planning method for creating a software solution. After determining the objective and parameters of software, software developers will design or hire designers to create a solution plan. It covers both the architectural view and low-level component and algorithm implementation difficulties.

Design is defined as "the process of using diverse approaches and principles to define a device, a process, or a system in sufficient detail to allow physical realization."

The design process converts the "what" of the requirements to the "how" of the design. Software design is the first step in SDLC (Software Design Life Cycle), which moves the concentration from problem domain to solution domain.

5.1 What is a Software Design?

A software design may be platform-independent or platform-specific, depending on the availability of the technology called for by the design.

Software design can be defined as the process of employing available capabilities to create a solution to the problem(s) at hand. As a result, the fundamental distinction between software analysis and design is that the outcome of a software problem analysis will be fewer problems to solve, and it should not change too much even if it is carried out by various team members or even whole different groups. However, because design is based on capabilities, several designs for the same problem can exist depending on the capabilities of the environment that will host the solution (whether it is some OS, web, mobile or even the new cloud computing paradigm). The solution will depend also on the used development environment (Whether you build a solution from scratch or using reliable frameworks or at least implement some suitable design patterns)

Software design is the process of transforming user requirements into a format that the programmer can use to code and implement software. An SRS (Software Requirement Specification) document is prepared for assessing user requirements; however more explicit and precise software requirements are required for coding and implementation. The output of this procedure can be directly implemented in computer languages.

Software design is the first step in SDLC (Software Design Life Cycle), which moves the concentration from problem domain to solution domain. It tries to specify how to fulfill the requirements mentioned in SRS.

5.2 Design Process

The phrase software design is often used to characterize the discipline that is also called software engineering—the discipline concerned with the construction of software that is efficient, reliable, robust, and easy to maintain.

Interface Design

Bringing Design to Software suggests that the design object is software, ignoring the interface devices that are the user's inextricable embodiment of software. Design cannot be easily split into software and device compartments: Physical interfaces both create and constrain software options. The great majority of computer applications currently offer themselves to users in a typical manner—a visual display with a keyboard and mouse. However, computers in the future will provide more resources for physical human-computer interactions. On a small scale, certain innovative devices are already in use.

Example: Pen-based personal digital assistants (PDAs), virtual-reality goggles and gloves, and computers embedded in electromechanical devices of all kinds. Researchers are exploring further possibilities, including tactile input and output devices, immersive environments, audio spaces, wearable computers, and a host of gadgets that bear little resemblance to today's personal computer or workstation. As experience with a wider variety of devices accumulates, the design of interaction based on new combinations of devices and software will be an important emerging topic in what we have—for the moment—called software design.

Human-computer Interaction

Whenever someone designs software that interacts with people, the effects of the design extend beyond the software itself to include the experiences that people will have in encountering and using that software. A person encountering any artifact applies knowledge and understanding, based on wide variety of cognitive mechanisms grounded in human capacities for perception, memory, and action. Researchers in human-computer interaction have studied the mental worlds of computer users, developing approaches and methods for predicting properties of the interactions and for supporting the design of interfaces. Although it would be overstating the case to say that the cognitive analysis of human-computer interaction has led to commonly accepted and widely applied methods, there is a substantial literature that can be of value to anyone designing interactive software.

Art

The experience of a person who is interacting with a computer system is not limited to the cognitive aspects that have been explored in the mainstream literature on human-computer interaction. As humans, we experience the world in aesthetic, affective, and emotional terms as well. Because computing evolved initially for use in the laboratory and the office, non-cognitive aspects have been

largely ignored, except by creators of computer games. Yet, whenever people experience a piece of software – whether it be a spreadsheet or a physics simulation – they have natural human responses. They experience beauty, satisfaction, and fun, or the corresponding opposites.

As computing becomes integrated into technologies for entertainment, and as the typical user moves from the well-regimented office to the home recreation room, software designers will need to focus more on the affective dimensions of human response. We can learn from the history of other human communication media, and can adapt the principles and techniques of novelists, film makers, composers, visual artists, and many other designers in what are loosely called the arts. Designing for the full range of human experience may well be the theme for the next generation of discourse about software design.

Perhaps even more difficult than the task of defining software is the task of defining design. A dictionary provides several loosely overlapping meanings, and a glance at the design section in a library or bookstore confuses the issue even more. Although we label it with a noun, design is not a thing. The questions that we can ask fruitfully are about the activity of designing.

Design is Conscious

People may refer to an object as being well designed whenever it is well suited to its environment, even if this suitability resulted from a process of unintentional evolution. In this book, we concentrate on what happens when a designer reflects on and brings focus to the design activity.

Consciousness about designing does not imply the application of a formal, consistent, or comprehensive theory of design or of a universal methodology. Systematic principles and methods at times may be applicable to the process of design, but there is no effective equivalent to the rationalized generative theories applied in mathematics and traditional engineering. Design consciousness is still pervaded by intuition, tacit knowledge, and gut reaction.

Design keeps Human Concerns in the Center

All engineering and design activities call for the management of tradeoffs. Real-world problems rarely have a correct solution of the kind that would be suitable for a mathematics problem or for a textbook exercise. The designer looks for creative solutions in a space of alternatives that is shaped by competing values and resource needs. In classical engineering disciplines, the tradeoffs can often be quantified: material strength, construction costs, rate of wear, and the like. In design disciplines, the tradeoffs are more difficult to identify and to measure. The designer stands with one foot in the technology and one foot in the domain of human concerns, and these two worlds are not easily commensurable.

As an example, it is easy for software designers to fall into a single-minded quest, in which ease of use (especially for beginning users) becomes a holy grail. But what is ease of use? How much does it matter to whom? A violin is extremely difficult for novices to play, but it would be foolhardy to argue that it should therefore be replaced by the autoharp. The value of an artifact may lie in high-performance use by virtuosos, or in ease of use for some special class of users, such as children or people with disabilities. There is room for the software equivalents of high strung racing cars alongside automatic-transmission minivans.

Design is a Dialog with Materials

The ongoing process of designing is iterative at two levels: iteration by the designer as a piece of current work develops and iteration by the community as successive generations reveal new possibilities for the medium. Designing is not a process of careful planning and execution, but rather it is like a dialog, in which the dialog partner – the designed object itself – can generate unexpected interruptions and contributions. The designer listens to the emerging design, as well as shapes it.

Design always implies a medium of construction, and new technologies bring with them new domains for design. Architecture as we know it appeared with the technology for building with stone. Graphic design emerged with the technologies of printing, and modern product design flourished with the development of plastics that expanded the possible variety of forms for everyday products.

Design is Creative

It is one thing to lay out a list of criteria for good design. It is quite another thing to do design well. Designer lacks the comforting restraints of a well-organized engineering discipline, because designing is inherently messy; it includes, but goes beyond, the ability to be creative in solving problems. It begins with creativity in finding the problems – envisioning the needs that people have but do not recognize.

Software Engineering Practices

Design is more an art than a science—it is spontaneous, unpredictable, and hard to define. The skill of the artist-designer is not reducible to a set of methods, and is not learned through the kind of structured curriculum that serves in science and engineering. On the other hand, it is not a mysterious gift. There is a long and rich tradition of education in the design fields; it draws on the interaction between learner and teacher, designer and critic.

Design is Communication

Previous sections have described the interaction between the user and his world, and the interaction between the designer and her materials. What matters most is the interaction between these two interactions. Virtuality is neither just what the designer thinks it is, nor what any particular user thinks it is. It exists in the ongoing dialog between designers and users. To succeed in designing, we need to understand how designers convey meaning.

At a surface level, designed artifacts communicate content. Skilled designers in every discipline know how to manage these layers of meaning. In addition, an active artifact—whether it be a computer program or a coffee maker—communicates to users about its use. A device as apparently simple as a door communicates to its users through convention. A door with a flat plate near shoulder level says “Push me!” One with a round knob says “Twist here.” Although these messages are constrained by the physical mechanisms, they are also a matter of convention and learning, as every tourist finds out in trying to deal with everyday objects in an unfamiliar culture.

As is true of any human language, what is visible in a statement is only a small part of the full picture. A hearer of the sentence “I’m going to kill you!” cannot interpret the intended meaning without knowing the situational context—was it said to a friend walking onto the tennis court, in an anonymous note mailed to the President, or by a parent to a child who has just left dirty clothing on the bathroom floor again? The literal meaning is but the shadow of the meaning in context.

The same is true for the artifacts that we build, including software: People do not approach them in isolation. Every object appears in a context of expectations that is generated by the history of previous objects and experiences, and by the surroundings in the periphery—the physical, social, and historical context in which the object is encountered.

Design has Social Consequences

Much of the discussion on software design uses examples from generic mass-distribution software, such as word processors, operating systems, spreadsheets, graphics programs, and games. Although many key concerns of software design are addressed in these applications, others are not. These highly visible applications are part of a larger picture that includes a vast number of vertical applications (for example, a medical-office billing application) and systems designed for a specific workplace setting. In these more targeted applications, the designer can see and take into account the specific effects of the design on the people who will inhabit it. Organizational aspects of software design in which we build integrated computer systems for specific organizations and workplaces. The needs and concerns of the people in those workplaces

can lead to complex—and, at times, controversial—design considerations, which we can address only by making the social and political dimensions an explicit part of the analysis and of the design dialog. The designer takes on a role that includes organizational as well as technical design, and can enlist workers directly in this process through techniques such as participatory design.

Design is a Social Activity

In concentrating on the activity of an individual designer, we can fall into the error of assuming that the overall quality of a design is primarily a result of the qualities and activities of the creative individual. As Kelley points out, the designer operates in a larger setting, which is both facilitated and constrained by interactions with other people.

5.3 Objectives of Software Design

Correctness: A good design should be correct i.e. it should correctly implement all the functionalities of the system.

Efficiency: A good software design should address the resources, time and cost optimization issues.

Understandability: A good design should be easily understandable, for which it should be modular and all the modules are arranged in layers.

Completeness: The design should have all the components like data structures, modules, and external interfaces, etc.

Maintainability: A good software design should be easily amenable to change whenever a change request is made from the customer side.

Flexibility: Able to modify on changing needs.

5.4 Software Design Levels

Interface Design: Interface design is the specification of the interaction between a system and its environment. This phase proceeds at a high level of abstraction with respect to the inner workings of the system

Attention is focused on the dialogue between the target system and the users, devices, and other systems with which it interacts.

- Precise description of events in the environment, or messages from agents to which the system must respond.
- Precise description of the events or messages that the system must produce. Specification on the data, and the formats of the data coming into and going out of the system.
- Specification of the ordering and timing relationships between incoming events or messages, and outgoing events or outputs.

Architectural Design - The architectural design is the highest abstract version of the system. It identifies the software as a system with many components interacting with each other. At this level, the designers get the idea of proposed solution domain.

- Gross decomposition of the systems into major components.
- Allocation of functional responsibilities to components.
- Component Interfaces
- Component scaling and performance properties, resource consumption properties, reliability properties, and so forth.
- Communication and interaction between components.

High-level Design- The high-level design breaks the 'single entity-multiple component' concept of architectural design into less-abstracted view of sub-systems and modules and depicts their interaction with each other.

High-level design focuses on how the system along with all of its components can be implemented in forms of modules. It recognizes modular structure of each sub-system and their relation and interaction among each other.

Detailed Design- Detailed design deals with the implementation part of what is seen as a system and its sub-systems in the previous two designs.

It is more detailed towards modules and their implementations. It defines logical structure of each module and their interfaces to communicate with other modules

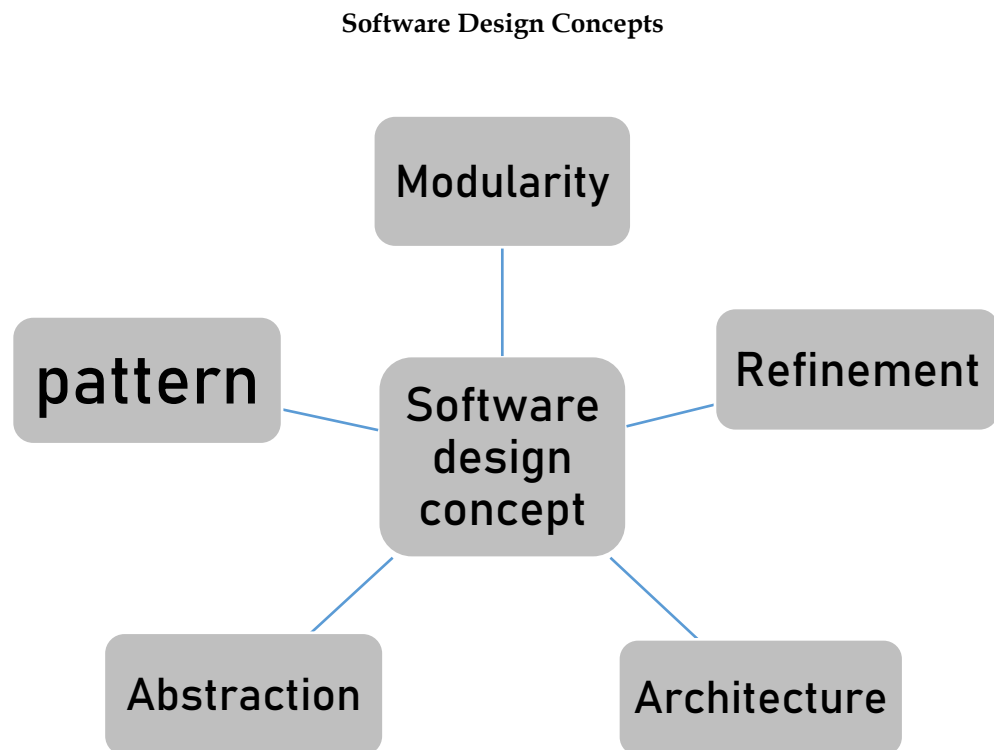
- Decomposition of major system components into program units.
- Allocation of functional responsibilities to units.
- User interfaces
- Unit states and state changes
- Data and control interaction between units
- Data packaging and implementation, including issues of scope and visibility of program elements
- Algorithms and data structures

5.5 Software Design Concepts

In order to deal with the complexity, software is designed in layers. When a programmer is worrying about the detailed design of one module, there are probably hundreds of other modules and thousands of other details that he cannot possibly worry about at the same time. For example, there are important aspects of software design that do not fall cleanly into the categories of data structures and algorithms. Ideally, programmers should not have to worry about these other aspects of a design when designing code.

The software design concept is the idea or principle behind the design. It describes how you plan to solve the problem of designing software, the logic, or thinking behind how you will design software

The software design concept provides a supporting and essential structure or model for developing the right software.



Modularization

Modularization is a technique to divide a software system into multiple discrete and independent modules, which are expected to be capable of carrying out task(s) independently. These modules may work as basic constructs for the entire software. Designers tend to design modules such that they can be executed and/or compiled separately and independently.

Modular design unintentionally follows the rules of 'divide and conquer' problem-solving strategy this is because there are many other benefits attached with the modular design of software.

Advantage of modularization:

- Smaller components are easier to maintain
- Program can be divided based on functional aspects
- Desired level of abstraction can be brought in the program
- Components with high cohesion can be re-used again.
- Concurrent execution can be made possible
- Desired from security aspect

Refinement

The refinement concept of software design is actually a process of developing or presenting the software or system in a detailed manner that means to elaborate a system or software.

Concurrency

Back in time, all software's were meant to be executed sequentially. By sequential execution we mean that the coded instruction will be executed one after another implying only one portion of program being activated at any given time. Say, software has multiple modules, and then only one of all the modules can be found active at any time of execution.

In software design, concurrency is implemented by splitting the software into multiple independent units of execution, like modules and executing them in parallel. In other words, concurrency provides capability to the software to execute more than one part of code in parallel to each other.

It is necessary for the programmers and designers to recognize those modules, which can be made parallel execution.

Example

The spell check feature in word processor is a module of software, which runs alongside the word processor itself.

On any software project of typical size, problems like these are guaranteed to come up. Despite all attempts to prevent it, important details will be overlooked. This is the difference between craft and engineering. Experience can lead us in the right direction. This is craft. Experience will only take us so far into uncharted territory. Then we must take what we started with and make it better through a controlled process of refinement. This is engineering.

As just a small point, all programmers know that writing the software design documents after the code instead of before, produces much more accurate documents. The reason is now obvious. Only the final design, as reflected in code, is the only one refined during the build/test cycle. The probability of the initial design being unchanged during this cycle is inversely related to the number of modules and number of programmers on a project.

In software engineering, we desperately need good design at all levels. In particular, we need good top level design. The better the early design, the easier detailed design will be. Designers should use anything that helps. Structure charts, Brooch diagrams, state tables, PDL, etc. — if it helps, then use it. We must keep in mind, however, that these tools and notations are not a software design. Eventually, we have to create the real software design, and it will be in some programming language. Therefore, we should not be afraid to code our designs as we derive them. We simply must be willing to refine them as necessary.

There is as yet no design notation equally suited for use in both top level design and detailed design. Ultimately, the design will end up coded in some programming language. This means that top level design notations have to be translated into the target programming language before detailed design can begin. This translation step takes time and introduces errors. Rather than translate from a notation that may not map cleanly into the programming language of choice, programmers often go back to the requirements and redo the top level design, coding it as they go. This, too, is part of the reality of software development.

5.6 Coupling and Cohesion

When a software program is modularized, its tasks are divided into several modules based on some characteristics. As we know, modules are set of instructions put together in order to achieve some tasks. They are though, considered as single entity but may refer to each other to work together. There are measures by which the quality of a design of modules and their interaction among them can be measured. These measures are called coupling and cohesion.

In modular approach tasks are divided into several modules based on some characteristics. Modules are set of instructions put together in order to achieve some tasks. There are measures by which the quality of a design of modules and their interaction among them can be measured. These measures are called coupling and cohesion.

Design is a two-part iterative process. First part is Conceptual Design that tells the customer what the system will do. Second is Technical Design that allows the system builders to understand the actual hardware and software needed to solve customer's problem.

Software Engineering Practices

Cohesion: Cohesion is a measure that defines the degree of intra-dependability within elements of a module. The greater the cohesion, the better is the program design.

A module having low coupling and high cohesion is said to be functionally independent of other modules

Modules with high cohesion tend to be preferable because high cohesion is associated with several desirable traits of software including robustness, reliability, reusability, and understandability.

Low cohesion is associated with undesirable traits such as being difficult to maintain, difficult to test, difficult to reuse, and even difficult to understand.

Types of Cohesion

Coincidental Cohesion (Worst)

Logical Cohesion

Temporal Cohesion

Procedural Cohesion

Communicational Cohesion

Sequential Cohesion

Functional Cohesion (Best)

- Co-incidental cohesion - It is unplanned and random cohesion, which might be the result of breaking the program into smaller modules for the sake of modularization. Because it is unplanned, it may serve confusion to the programmers and is generally not-accepted.
- Logical cohesion - When logically categorized elements are put together into a module, it is called logical cohesion.
- Temporal Cohesion - When elements of module are organized such that they are processed at a similar point in time, it is called temporal cohesion.
- Procedural cohesion - When elements of module are grouped together, which are executed sequentially in order to perform a task, it is called procedural cohesion.
- Communicational cohesion - When elements of module are grouped together, which are executed sequentially and work on same data (information), it is called communicational cohesion.
- Sequential cohesion - When elements of module are grouped because the output of one element serves as input to another and so on, it is called sequential cohesion.
- Functional cohesion - It is considered to be the highest degree of cohesion, and it is highly expected. Elements of module in functional cohesion are grouped because they all contribute to a single well-defined function. It can also be reused.

Coupling

Coupling is a measure that defines the level of inter-dependability among modules of a program. It tells at what level the modules interfere and interact with each other. The lower the coupling, the better the program the use of global variables can result in an enormous number of connections between the modules of a program

The degree of coupling between two modules is a function of several factors

How complicated the connection is

Whether the connection refers to the module itself or something inside it

What is being sent or received

Coupling can be "low" ("loose" and "weak") or "high" ("tight" and "strong").

Low coupling means that one module does not have to be concerned with the internal implementation of another module, and interacts with another module with a stable interface

Low coupling is a sign of a well-structured computer system.

Types of coupling

Data coupling

Content coupling

Common coupling

Control coupling

Stamp coupling

External Coupling

- **Content coupling** - When a module can directly access or modify or refer to the content of another module, it is called content level coupling.
- **Common coupling**- When multiple modules have read and write access to some global data, it is called common or global coupling.
- **Control coupling**- Two modules are called control-coupled if one of them decides the function of the other module or changes its flow of execution.
- **Stamp coupling**- When multiple modules share common data structure and work on different part of it, it is called stamp coupling. Stamp coupling is when modules share a composite data structure, each module not knowing which part of the data structure will be used by the other

Example: passing a student record to a function which calculates the student's GPA

- **Data coupling**- Data coupling is when two modules interact with each other by means of passing data (as parameter). If a module passes data structure as parameter, then the receiving module should use all its components.
- **External coupling**- External coupling occurs when two modules share an externally imposed data format, communication protocol, or device interface.

Ideally, no coupling is considered to be the best.

5.7 Data Flow Diagram and Flow Chart

"A picture is worth a thousand words"

Data flow diagram is graphical representation of flow of data in an information system.

It includes data inputs and outputs, data stores, and the various sub processes the data moves through.

Diagrams are used to map out an existing system and make it better or to plan out a new system for implementation. Visualizing each element makes it easy to identify inefficiencies and produce the best possible system.

DFDs are built using standardized symbols and notation to describe various entities and their relationships.

Types of DFD

Logical

Physical

Logical

Logical data flow diagrams focus on what happens in a particular information flow. What information is being transmitted, what entities are receiving that info, what general processes occur, etc. A logical DFD doesn't deal with the technical aspects of a process or system.

Physical

Software Engineering Practices

Physical data flow diagrams focus on how things happen in an information flow. Physical DFD specify the software, hardware, files, and people involved in an information flow. A detailed physical data flow diagram can facilitate the development of the code needed to implement a data system.

Levels of Data Flow Diagram

Level 0

Level 1

Level 2

Level 0 DFD

It is also known as a context diagram. It's designed to be an abstraction view, showing the system as a single process with its relationship to external entities. It represents the entire system as a single bubble with input and output data indicated by incoming/outgoing arrows.

Level 1 DFD

In level 1 DFD, the context diagram is decomposed into multiple modules/processes. In this level, we highlight the main functions of the system and breakdown the high-level process of 0-level DFD into sub processes.

Level 2 DFD

At this level, DFD shows how data flows inside the modules mentioned in Level 1. It can be used to plan or record the specific necessary detail about the system functioning.

DFD symbols

There are four basic elements of a data flow diagram:

Processes

Data stores

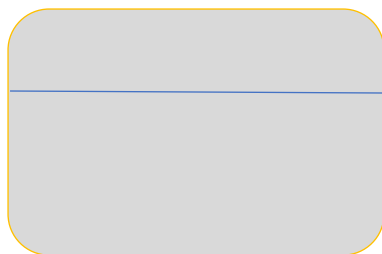
External entities

Data flows

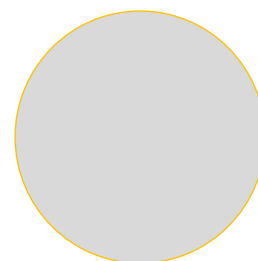
Depending on the methodology, DFD symbols vary slightly.

Process

A process receives input data and produces output with a different content or form. Processes can be as simple as collecting input data and saving in the database



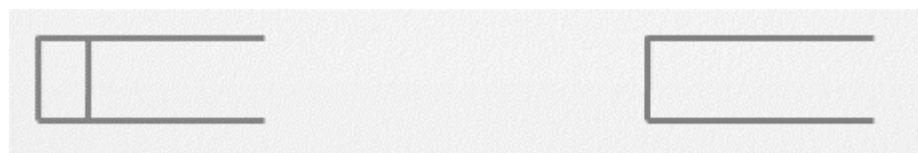
Gane and Sarson



Yourdon and Coad

Data store

Data stores are repositories of data in the system. They are sometimes also referred to as files.

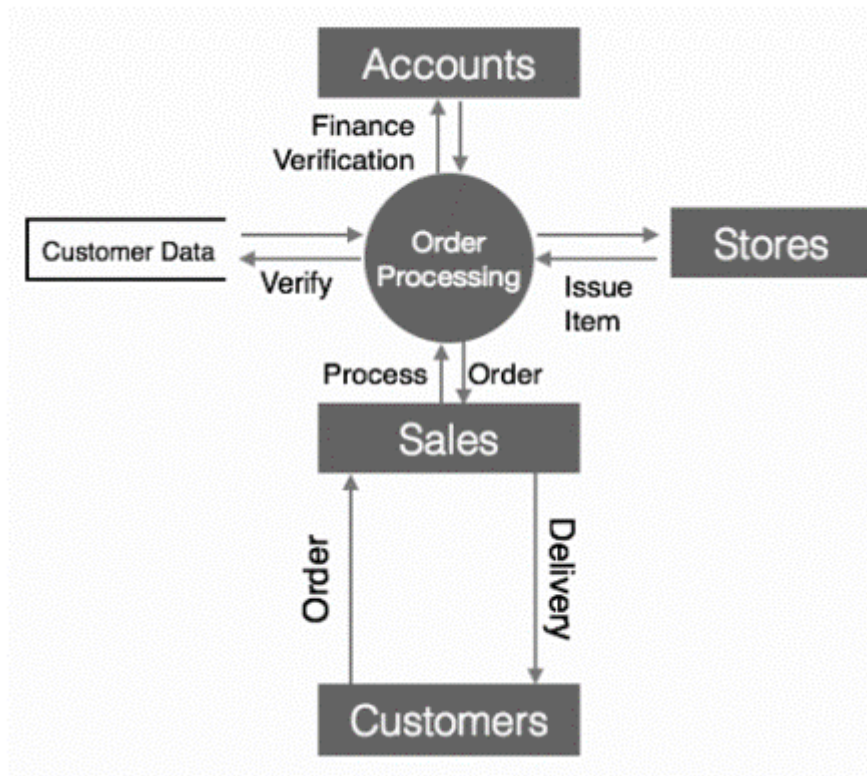
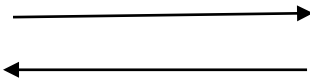


External entities

External entities are objects outside the system, with which the system communicates. External entities are sources and destinations of the system's inputs and outputs

**Data flow**

A data-flow is a path for data to move from one part of the information system to another

**Guideline to create a DFD**

- Identify major inputs and outputs in your system
- Build a context diagram
- Expand the context diagram into a level 1 DFD
- Expand to a level 2+ DFD
- Confirm the accuracy of your final diagram
- The name of the entity should be easy and understandable without any extra assistance
- The processes should be numbered or put in ordered list to be referred easily.

- The DFD should maintain consistency across all the DFD levels.

Advantages of DFD

- It helps us to understand the functioning and the limits of a system.
- It is a graphical representation which is very easy to understand as it helps visualize contents.
- Data Flow Diagram represent detailed and well explained diagram of system components.
- It is used as the part of system documentation file.

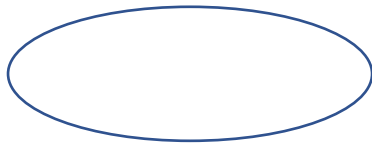
5.8 Flow Chart

A flow chart is a graphical or symbolic representation of a process. Each step in the process is represented by a different symbol and contains a short description of the process step. The flow chart symbols are linked together with arrows showing the process flow direction.

Flow chart symbols

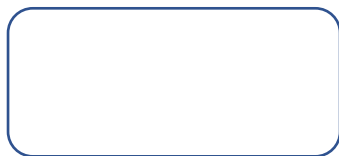
Terminator

The terminator symbol represents the starting or ending point of the system.



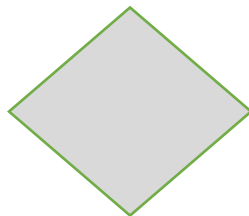
Process

A rectangular flow chart shape indicating a normal process flow step



Decision

A diamond flow chart shape indicating a branch in the process flow. Lines' coming out from the diamond indicates different possible situations, leading to different sub-processes.



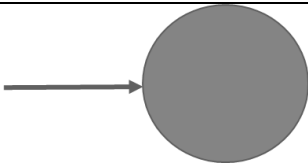
Flow lines

Flow lines are used to show flow of data



Connector

A small, labeled, circular flow chart shape used to indicate a jump in the process flow.

**Data**

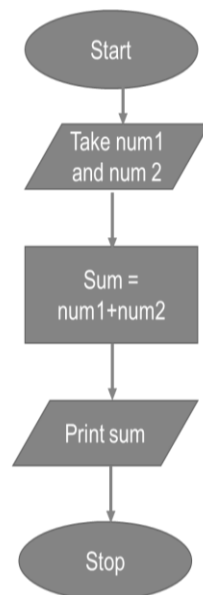
It represents information entering or leaving the system. An input might be an order from a customer. Output can be a product to be delivered.

**Document**

Used to indicate a document or report



Addition of two numbers.

**5.9 Architectural Design**

Design is a process in which representations of data structure, program structure, interface characteristics, and procedural details are synthesized from information requirements

The initial design process of identifying these sub-systems and establishing a framework for sub-system control and communication is called architectural design

In architectural design process the activities carried out are system structuring (system is decomposed into sub-systems and communications between them are identified), control modeling, modular decomposition

Software Engineering Practices

IEEE defines architectural design as “the process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system.”

Architectural Design Representation

Structural model: Illustrates architecture as an ordered collection of program components

Dynamic model: Specifies the behavioral aspect of the software architecture and indicates how the structure or system configuration changes as the function changes due to change in the external environment

Process model: Focuses on the design of the business or technical process, which must be implemented in the system

Functional model: Represents the functional hierarchy of a system

Framework model: Attempts to identify repeatable architectural design patterns encountered in similar types of application. This leads to an increase in the level of abstraction

Architectural styles

An architecture style involves its components, connectors, and constraints on combining components.

Pipes and Filters

In Pipes and Filters, each component (filter) reads streams of data on its inputs and produces streams of data on its output. Pipes are the connectors that transmit output from one filter to another. Functional transformations process their inputs to produce outputs

Call-and-return systems

The program structure decomposes function into a control hierarchy where a “main” program invokes (via procedure calls) a number of program components, which in turn may invoke still other components.

Example: Structure Chart is a hierarchical representation of main program and subprograms

Object-oriented systems

In Object-oriented systems, component is an encapsulation of data and operations that must be applied to manipulate the data. Communication and coordination between components is accomplished via message calls.

Layered systems

In Layered systems, each layer provides service to the one outside it, and acts as a client to the layer inside it. They are arranged like an “onion ring”. E.g. OSI ISO model

Data - centered systems

A data store will reside at the center of this architecture and is accessed frequently by the other components that update, add, delete or modify the data present within the store.

Data centered systems use repositories. Repository includes a central data structure representing current state, and a collection of independent components that operate on the central data store

Distributed systems

Distributed system architecture is the Client/Server where a server system responds to the requests for actions / services made by client systems. Clients access server by remote procedure call.

Set of stand-alone servers which provide specific services such as printing, data management, etc.

5.10 Component Based Design

In component-based software engineering, a software system is considered as a set of software components assembled together instead of as a set of functions from the traditional perspective. This architecture focuses on the decomposition of the design into logical components which contain events, methods and properties.

Component-Based architecture divides the problem into sub-problems and each problem associated with component partitions. It provides a higher level of abstraction than object-oriented design principles. It does not focus on issues such as communication protocols and shared state.

The objective of component-based architecture is to ensure component reusability. A component encapsulates functionality and behavior of a software element into a reusable and self-deployable binary unit.

Key Issues of Component based design

Component planning

Component building

Component assembling

Component representation

Component retrieval

A component is a modular, portable, replaceable, and reusable set of well-defined functionality that encapsulates its implementation and exporting it as a higher-level interface. A software component is a unit of composition with contractually specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to third-party composition.

Characteristics of Components

Reusability – Components are usually designed to be reused in different situations in different applications. However, some components may be designed for a specific task.

Replaceable – Components may be freely substituted with other similar components.

Not context specific – Components are designed to operate in different environments and contexts.

Extensible – A component can be extended from existing components to provide new behavior.

Encapsulated – A component depicts the interfaces, which allow the caller to use its functionality, and do not expose details of the internal processes or any internal variables or state.

Independent – Components are designed to have minimal dependencies on other components.

Component Based design activities

Component Qualification: This activity ensures that the system architecture defines the requirements of the components for becoming a reusable component. Reusable components are generally identified through the traits in their interfaces.

Component Adaptation: This activity ensures that the architecture defines the design conditions for all component and identifying their modes of connection.

Component Composition: This activity ensures that the Architectural styles of the system integrates the software components and form a working system. By identifying connection and coordination mechanisms of the system, the architecture describes the composition of the end product.

Advantages of Component Based design

Reusable – The use of reusable components means that they can be used to spread the development and maintenance cost across several applications or systems.

Reduced cost – the use of third-party components allows you to spread the cost of development and maintenance

Ease of development – Components implement well-known interfaces to provide defined functionality, allowing development without impacting other parts of the system.

Ease of deployment – as new compatible versions become available, it is easier to replace existing versions with no impact on the other components or the system as a whole.

Modification of technical complexity – A component modifies the complexity through the use of a component container and its services.

Software Engineering Practices

Reliability – The overall system reliability increases since the reliability of each individual component enhances the reliability of the whole system via reuse.

Independent – Independency and flexible connectivity of components Independent development of components by different group in parallel Productivity for the software development and future software development

5.11 Object-oriented design

Object oriented design works around the entities and their characteristics instead of functions involved in the software system. This design strategy focuses on entities and its characteristics. The whole concept of software solution revolves around the engaged entities.

Object oriented design represent the problem domain, makes it easy to understand and produce design. In the object-oriented design, the system is viewed as a collection of objects (i.e., entities). The state is distributed among the objects, and each object handles its state data.

It is the discipline of defining the objects and their interactions to solve a problem that was identified and documented during the Object Oriented Analysis (OOA). An analysis model created using object oriented analysis is transformed by object oriented design into a design model that works as a plan for software development.

Object Oriented Design concepts

Objects - All entities involved in the solution design are known as objects. For example, person, banks, company and customers are treated as objects. Every entity has some attributes associated to it and has some methods to perform on the attributes.

Classes - A class is a generalized description of an object. An object is an instance of a class. Class defines all the attributes, which an object can have and methods, which defines the functionality of the object.

Encapsulation - In OOD, the attributes (data variables) and methods (operation on the data) are bundled together is called encapsulation. Encapsulation not only bundles important information of an object together, but also restricts access of the data and methods from the outside world. This is called information hiding.

Inheritance - OOD allows similar classes to stack up in hierarchical manner where the lower or sub-classes can import, implement and re-use allowed variables and methods from their immediate super classes. This property of OOD is known as inheritance. This makes it easier to define specific class and to create generalized classes from specific ones.

Messages: Objects communicate by message passing. Messages consist of the integrity of the target object, the name of the requested operation, and any other action needed to perform the function. Messages are often implemented as procedure or function calls.

Abstraction In object-oriented design, complexity is handled using abstraction. Abstraction is the removal of the irrelevant and the amplification of the essentials.

Polymorphism - OOD languages provide a mechanism where methods performing similar tasks but vary in arguments, can be assigned same name. This is called polymorphism, which allows a single interface performing tasks for different types. Depending upon how the function is invoked, respective portion of the code gets executed.

Characteristics of Object Oriented Design

- Objects are abstractions of the real-world or system entities and manage themselves.
- The objects are independent and in an encapsulated state and representation information.
- Shared data areas are eliminated.
- System functionality is expressed in terms of object services.
- Communication between objects is through message passing.
- The objects may be distributed and may execute sequentially or in parallel.

Design Process

A solution design is created from requirement or previous used system and/or system sequence diagram. Objects are identified and grouped into classes on behalf of similarity in attribute

characteristics. Class hierarchy and relation among them are defined. Application framework is defined.

Advantages of Object Oriented Design

- Easier to maintain objects.
- Objects may be understood as stand-alone entities.
- Objects are appropriate reusable components.
- For some systems, there may be an obvious mapping from real entities to system objects.

Top down Design approach

Top-down design takes the whole software system as one entity and then decomposes it to achieve more than one sub-system or component based on some characteristics.

Each subsystem or component is then treated as a system and decomposed further. This process keeps on running until the lowest level of system in the top-down hierarchy is achieved.

Bottom-up Design

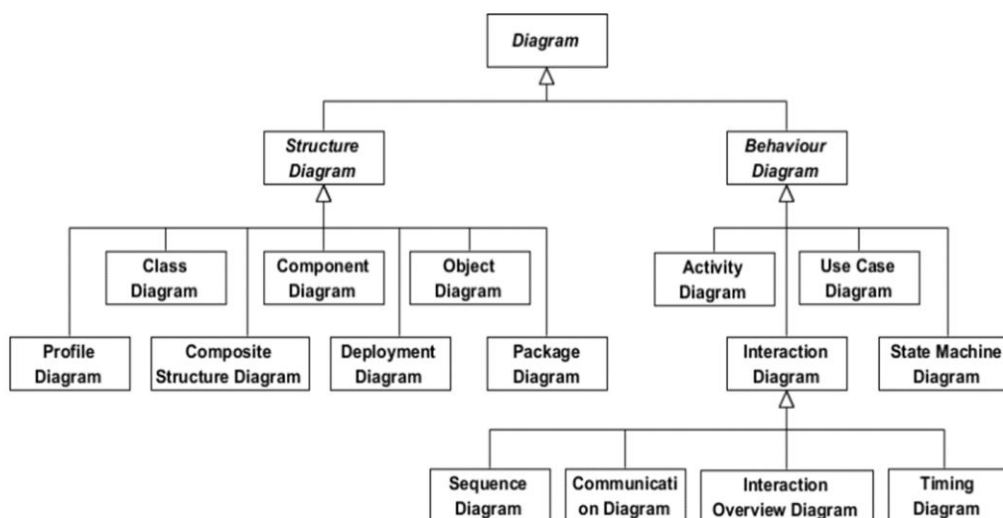
The bottom up design model starts with most specific and basic components. It proceeds with composing higher level of components by using basic or lower level components.

It keeps creating higher level components until the desired system is not evolved as one single component. With each higher level, the amount of abstraction is increased.

5.12 Use Case Diagram, Class Diagram and Activity Diagram

Unified Modeling Language is a standardized modeling language consisting of an integrated set of diagrams, developed to help system and software developers for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modeling and other non-software systems.

UML diagram



A structure diagram represents the static aspects of the system. It emphasize the things that must be present in the system being modeled

Behaviour diagrams represent the dynamic aspect of the system. It emphasizes what must happen in the system being modeled.

A Unified Modeling Language (UML) use case diagram is the primary form of system/software requirements for a new software program underdeveloped. A use case diagram is a dynamic or behaviour diagram in UML. Use case diagrams model the functionality of a system using actors and

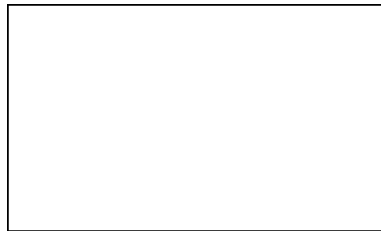
Software Engineering Practices

use cases. Use cases are a set of actions, services, and functions that the system needs to perform. Use case diagrams are used to gather the requirements of a system including internal and external influences. These requirements are mostly design requirements.

Symbols and Notations

System

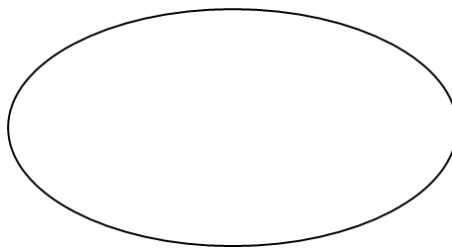
Draw your system's boundaries using a rectangle that contains use cases. Place actors outside the system's boundaries.



System

Use case

A description of a set of sequences of actions, including variants, that system performs that yields an observable value to an actor.

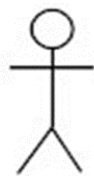


Use case

Actor

The people or systems that provide or receive information from the system; they are among the stakeholders of a system.

It could be human beings, other systems, timers and clocks or hardware devices. Actors that stimulate the system and are the initiators of events are called primary actors (active). Actors that only receive stimuli from the system are called secondary actors (passive).

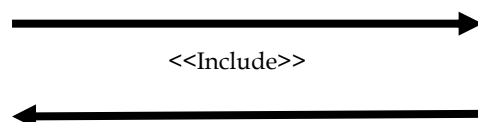


Actor

Relationship

Include: Specifies that the source use case explicitly incorporates the behavior of another use case at a location specified by the source.

Extend: Specifies that the target use case extends the behavior of the source.



<<Extend>>

Guidelines for use case diagram

- Use cases should ideally begin with a verb – i.e generates report. Use cases should NOT be open ended – i.e Register (instead should be named as Register New User)
- Avoid showing communication between actors.
- Actors should be named as singular. i.e student and NOT students. NO names should be used – i.e John, Sam, etc.
- Do NOT show behaviour in a use case diagram; instead only depict only system functionality.
- Use case diagram does not show sequence – unlike DFDs.

Class Diagram

Class diagram is a static diagram. It represents the static view of an application

It describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects.

The class diagrams are widely used in the modeling of object oriented systems

Class

A class represents a concept which encapsulates state (attributes) and behaviour (operations).

A Class is a blueprint for an object. Objects and classes go hand in hand

Class describes what an object will be, type of objects, while objects are usable instances of classes

Class Notation

Class Name: The name of the class appears in the first partition.

Class Attributes

Class Operations (Methods)

Example student (class)

Attributes (name, roll no, height)

Methods (playing, reading)

Class Attributes:

Attributes are shown in the second partition.

The attribute type is shown after the colon.

Attributes map onto member variables (data members) in code.

Class Operations (Methods):

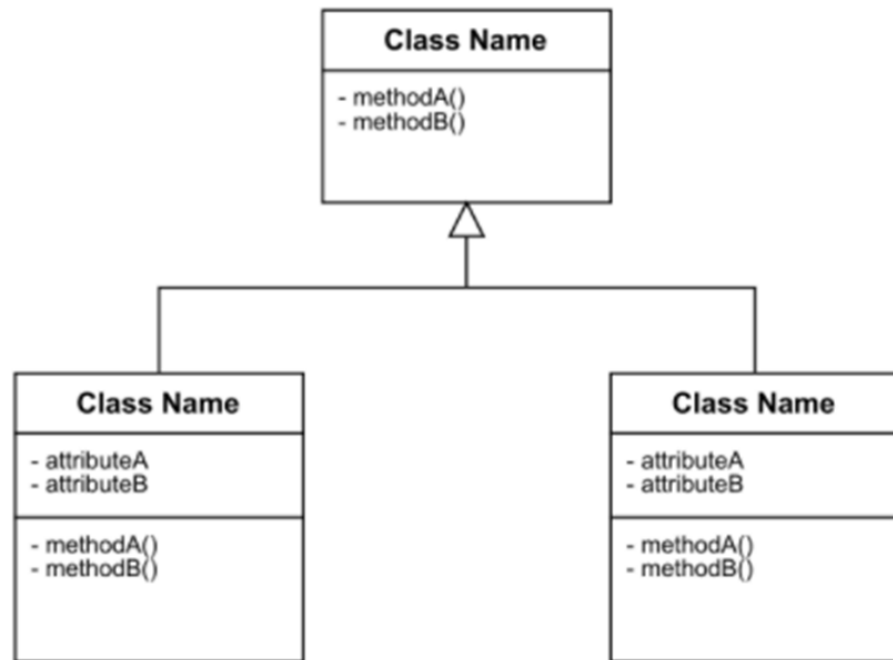
Operations are shown in the third partition. They are services the class provides.

The return type of a method is shown after the colon at the end of the method signature.

The return type of method parameters are shown after the colon following the parameter name.

Operations map onto class methods in code.

Class diagram



Advantages of class diagram

- It can represent the object model for complex systems.
- It provides a general schematic of an application for better understanding.
- It represents a detailed chart by highlighting the desired code, which is to be programmed.
- It reduces the maintenance time by providing an overview of how an application is structured before coding.
- It is helpful for the stakeholders and the developers.

Activity diagram

In UML activity diagram to describe dynamic aspects of the system

Activity diagram represent the flow from one activity to another activity. The activity can be described as an operation of the system.

An activity diagram is a behavioral diagram i.e. it depicts the behavior of a system.

Objective of Activity Diagram

- Modeling work flow by using activities.
- Modeling business requirements.
- Model workflows between/within use cases
- High level understanding of the system's functionalities.
- Investigating business requirements at a later stage

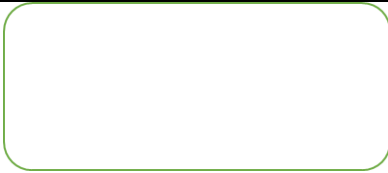
Activity Diagram Notations

Initial State or Start Point: A small filled circle followed by an arrow represents the initial action state or the start point for any activity diagram.



Start/ initial point

Activity or Action State: An action state represents the non-interruptible action of objects.



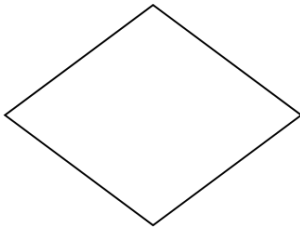
Action/ activity

Action Flow: Action flows, also called edges and paths, illustrate the transitions from one action state to another.



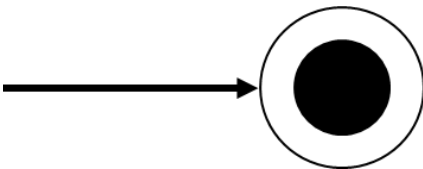
Action flow

Decisions and Branching: A diamond represents a decision with alternate paths. When an activity requires a decision prior to moving on to the next activity, add a diamond between the two activities.



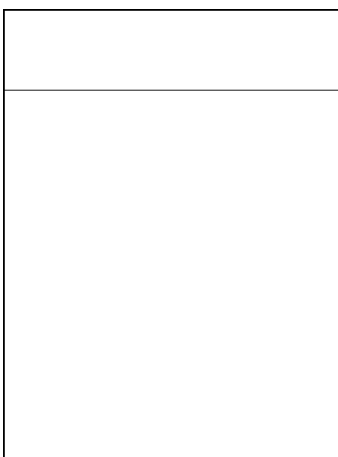
Decision box

Final State or End Point: An arrow pointing to a filled circle nested inside another circle represents the final action state



End point

Swim lanes – We use Swim lanes for grouping related activities in one column. Swim lanes group related activities into one column or one row.



Swim lanes

Guidelines for activity diagram

- All activities in the system should be named.
- Activity names should be meaningful.
- Constraints must be identified.
- Activity associations must be known.

Use case diagram Vs. Activity diagram

- Activity diagram is used to model the workflow depicting conditions, constraints, sequential and concurrent activities
- Use Case is to just depict the functionality i.e. what the system does and not how it is done
- Activity diagram shows 'How' while a Use case shows 'What' for a particular system

Summary

- Whenever objects are created for people to use, design is pervasive. Design may be done methodically or offhandedly, consciously or accidentally.
- The phrase software design is often used to characterize the discipline that is also called software engineering – the discipline concerned with the construction of software that is efficient, reliable, robust, and easy to maintain.
- Software design principles represent a set of guidelines that helps us to avoid having a bad design.
- The design principles are associated to Robert Martin who gathered them in “Agile Software Development: Principles, Patterns, and Practices”.
- In order to deal with the complexity, software is designed in layers. When a programmer is worrying about the detailed design of one module, there are probably hundreds of other modules and thousands of other details that he cannot possibly worry about at the same time.
- A modular design reduces the design complexity and results in easier and faster implementation by allowing parallel development of different parts of a system.

Keywords

Coupling: Coupling is an internal software attribute that can be used to indicate the degree of interdependence among the components of a software system.

Dependency Inversion Principle: Dependency Inversion Principle states that we should decouple high level modules from low level modules.

Design Patterns: A software design pattern describes a family of solutions to a software design problem.

Modular Design: A modular design reduces the design complexity and results in easier and faster implementation by allowing parallel development of different parts of a system.

Self Assessment

1. Software design process is ____
 - A. Converts the “what” of the requirements to the “how” of the design
 - B. The first step in SDLC (Software Design Life Cycle)
 - C. Process of problem solving and planning for a software solution
 - D. All of above

2. Which is not Software Design Level?
 - a. Interface Design
 - b. Architectural Design
 - c. Use case design.
 - d. High-level Design

3. Software Design Concept include____
 - A. Modularity
 - B. Refinement
 - C. Abstraction
 - D. All of above

4. How to measure quality and interaction between modules
 - A. Coupling
 - B. Cohesion
 - C. Both coupling and cohesion
 - D. None of above

5. Which is not a type of coupling?
 - A. Content
 - B. Common
 - C. Logical
 - D. Control

6. Which is not a type of cohesion?
 - A. Temporal
 - B. Procedural
 - C. Communicational
 - D. Stamp

7. What are the types of Data Flow Diagram?
 - A. Physical
 - B. Logical
 - C. Both physical and logical
 - D. None of above

8. Which is not a part of DFD symbol?
 - A. Processes
 - B. Data stores
 - C. Decision table
 - D. External entities

9. Use case and activity diagram is part of ____
 - A. Structure diagram

- B. Behavior diagram
 - C. Both Behavior diagram and Structure diagram
 - D. None of above
-
- 10. Which symbol is not a part of use case diagram?
 - A. Actor
 - B. Decision making
 - C. System
 - D. Use case

Answer for Self Assessment

- | | | | | |
|------|------|------|------|-------|
| 1. D | 2. C | 3. D | 4. C | 5. C |
| 6. D | 7. C | 8. C | 9. B | 10. B |

Review Questions

1. What is significance of good design?
2. Explain with example that how the modular design reduces the design complexity?
3. "Software Design is more an art than a science." Analyze.
4. Clients should not be forced to depend upon interfaces that they don't use. Explain.
5. Discuss the set of guidelines provided by the software design principles that helps to avoid having a bad design.
6. Dependency Inversion and Inversion of Control have the same meaning? Why or why not? Explain briefly.
7. A good design is the one that has low coupling. Explain.
8. Engineering is more about how you do the process than it is about what the final product looks like. Discuss.
9. The design of software is often depicted by graphs that show components and their relationships. Comment.
10. "Two modules that are tightly coupled are strongly dependent on each other." Explain why?

**Further Reading**

- Rajib Mall, Fundamentals of Software Engineering, Second Edition, PHI.
- Richard Fairpy, Software Engineering Concepts, Tata McGraw Hill, 1997.
- R.S. Pressman, Software Engineering a Practioner's Approach, 5th edition, Tata McGraw
- Hill Higher education.
- Sommerville, Software Engineering, Sixth edition, Pearson Education.



- http://en.wikipedia.org/wiki/Software_design
- http://www.developerdotstar.com/mag/articles/reeves_design.html
- <https://www.geeksforgeeks.org/>
- <https://www.tutorialspoint.com/>